

Rapid heat source layout optimization in three-dimensional integrated circuits using artificial neural network reduced-order model in combination with Bayesian optimization

Haitao Zhang¹ | Jianhao Song¹ | Xixin Rao¹  |
Huizhong Liu¹ | Chengdi Xiao^{1,2}

¹School of Advanced Manufacturing, Nanchang University, Nanchang, China

²Shanghai Highly Electric Co., Ltd., Shanghai, China

Correspondence

Chengdi Xiao, School of Advanced Manufacturing, Nanchang University, Nanchang, China.

Email: xcd0719@ncu.edu.cn

Funding information

Natural Science Foundation of China, Grant/Award Number: 52006095

Abstract

In this study, an efficient optimization framework was developed to determine the parameters of through-silicon vias and the layout of heat sources in three-dimensional integrated circuits (3D ICs), employing an artificial neural network (ANN) reduced-order model in conjunction with a Bayesian optimization (BO) algorithm. The proposed method effectively predicts the temperature distribution in 3D ICs and refines their thermal parameters, offering solutions to thermal management challenges. Latin hypercube sampling was utilized for data sampling, enhancing the previously established rapid thermal analysis method through parameterization of heat source locations. The temperature distribution data for varying hotspot locations in 3D ICs were fitted using an appropriately defined objective function, leading to the development of a reduced-order ANN model that accelerates temperature prediction. The computational results demonstrate that the neural network model exhibits a deviation in predicted values of less than 2%, and the coefficient of determination R^2 approximately 0.93, underscoring high predictive accuracy. Additionally,

the optimization outcomes and the efficiency of the selected BO algorithm were thoroughly evaluated. Notably, the BO algorithm achieved the global optimum in just 4.07 s across 250 iterations, demonstrating an effective power distribution strategy for the 3D ICs model.

KEYWORDS

Bayesian optimization, feedforward neural networks, thermal design optimization, three-dimensional integrated circuits

1 | INTRODUCTION

Three-dimensional integrated circuits (3D ICs) incorporate a significant array of through silicon vias (TSVs) and hierarchical packaging architectures, culminating in highly integrated 3D ICs configurations. Such configurations introduce intrinsic thermal management challenges.^{1–3} Addressing thermal management issues pertinent to 3D ICs necessitates the optimization of TSVs geometric parameters and strategic placement of heat sources. Given that one end of 3D ICs is typically dedicated to electrical interconnections, the opposing end becomes the sole avenue for heat dissipation, thereby constraining the dissipation pathways. Heat conduction via TSVs emerges as a pivotal heat transfer conduit, underscoring the importance of TSVs optimization and methodical heat source placement. Strategically positioning heat sources within 3D ICs, coupled with the enhanced heat dissipation afforded by the superior thermal conductivity of TSVs, is theoretically significant for thermal regulation in these circuits. In practice, the optimization challenges evolve into complex, high-dimensional, nonlinear, and multiextremal problems that are computationally intensive and resource-demanding. Constructing a predictive model from pre-existing training data and utilizing this model for optimization can substantially decrease the execution time of optimization algorithms. Conventional machine learning algorithms encompass decision trees, random forests, support vector machines, and neural networks.⁴ The neural network was used for training and validation of the amassed data, with the application of neural networks as reduced-order models being well-established in thermal analysis.^{4,5} Artificial neural networks (ANN) obviate the necessity of formulating mathematical equations between inputs and outputs, operating as black-box models that generate corresponding mathematical representations through the training, testing, and validation of input data.^{6–8} Feedforward neural networks, a specific type of neural network model, can establish corresponding mathematical models with a limited number of sample points, fit the nonlinear relationship between inputs and outputs, and swiftly obtain the required objective function values. Employing feedforward neural network-based reduced-order models can efficaciously mitigate the computational resource expenditure inherent in numerical simulations.

In the thermal analysis of 3D ICs, the optimization of material property parameters and the distribution of heat sources presents a significant computational challenge. Moreover, robust data support is essential for the thermal management optimization of TSVs. The complexity of factors influencing heat source distribution and the extensive solution space render the development of an

accurate mathematical model a formidable task. As a result, current thermal management optimization strategies predominantly utilize heuristic algorithms, such as Simulated Annealing (SA), Ant Colony Optimization (ACO), Particle Swarm Optimization (PSO), and Genetic Algorithms (GA), referenced in studies.^{9–12} These algorithms are advantageous as they primarily compute objective function values without stringent requirements on the nature of the optimization problem, such as convexity, differentiability, and Lipschitz continuity of the objective function's derivative. Ni et al.¹³ introduced a temperature-aware layout planning algorithm for fixed-contour 3D ICs, leveraging simulated annealing. This approach strategically positions the ICs' hotspots on the bottom layer, closer to the heat sink, and imposes judicious thermal constraints within and between layers to minimize the peak temperature of the 3D ICs. Ren et al.¹⁴ developed an analytical model for TSV units, employing the Hasselman thermal conductivity extraction model to calculate the equivalent thermal conductivity of TSVs in both the x - y plane and the z -direction. Building on this, they proposed a thermal TSV layout optimization algorithm, also based on simulated annealing, which negotiates the balance between peak temperature, wire length, and layout planning area.

However, traditional heuristic algorithms are commonly used in thermal optimization, their efficiency is often limited, and they can be computationally intensive. The integration of Bayesian optimization, which employs probabilistic theory to manage uncertainty in estimates, has markedly improved optimization efficacy. This method's ability to enhance performance has led to its widespread adoption for hyperparameter tuning, and it has been increasingly applied to thermal analysis of chips in recent years. Otaki et al.¹⁵ utilized a lumped capacitance thermal network model in tandem with Bayesian optimization (BO) to refine the thermal design of transient heating in chip electronic circuit board layouts. Wang et al.¹⁶ explored the thermal layout optimization of multichip modules using a convolutional neural network as a reduced-order model, which served as a surrogate to the traditional finite element model, in conjunction with BO. Park et al.¹⁷ applied machine learning techniques and BO to streamline the selection of material property parameters and materials for 3D ICs, achieving a notable decrease in the consumption of computational resources. Although BO has been demonstrated to significantly reduce resource usage in the thermal management optimization process, its application alongside conventional numerical simulation solvers is not without challenges. Each iteration of parameter adjustment within the optimization process requires re-evaluation of the numerical model, leading to considerable computational demands. Furthermore, software limitations may restrict the freedom to select new search points during each iteration of the calculation. To address these limitations, the implementation of a feedforward neural network as a reduced-order model, informed by data from finite element analyses, can provide an effective solution.

In this study, we have harnessed the synergy of ANN reduced-order model and BO to enhance the thermal management of a three-layer stacked 3D ICs model. The predictive accuracy of the ANN reduced-order model was validated by comparison with finite element analysis results. The optimization performance was further benchmarked against established algorithms, such as GA and surrogate optimization techniques. The devised objective function, which normalizes the maximum temperature, average temperature, and maximum temperature gradient, facilitates the rapid prediction of these thermal metrics within the 3D ICs model under consistent material parameters, albeit with variable heat source distributions. BO, in this framework, expedites the discovery of optimal heat source distribution with a reduced number of iterations. Compared to the utilization of BO in isolation, the integrated approach with neural network predictive models enables the swift computation of optimization objective function values with a predefined data set. Moreover, the parameterization of heat source

positioning augments the adaptability of the model, allowing for more nuanced selection of heat source locations. This adaptability is crucial for incorporating constraints that reflect the practical considerations of actual chip design scenarios.

2 | METHODOLOGY

2.1 | Model setup

In this research, we initially established a finite element simulation model of stacked 3D ICs and subsequently imported it into COMSOL Multiphysics software to simulate the steady-state temperature field distribution. The simulation outputs were then utilized to train the ANN model. Building upon our prior work, which presented a methodology for rapidly determining the steady-state temperature distribution of 3D ICs,¹⁸ we constructed a stacked 3D ICs model as illustrated in Figure 1. This model comprises a three-layer chip assembly atop a silicon interposer, designated as Die1, Die2, and Die3. Each layer's dimensions are $400\text{ }\mu\text{m} \times 400\text{ }\mu\text{m}$ and includes a 10×10 array of uniformly distributed TSVs, interconnected via microbumps encapsulated with underfill. The TSVs feature a pitch of $40\text{ }\mu\text{m}$, a volume fraction of $\pi/25$, and a slope angle of 6° .

It was assumed that the chip's upper surface was affixed to a water-cooling radiator, while the bottom adapter plate interfaced with the ambient air. The remaining wall boundaries were encased with silica, effectively minimizing heat exchange with the surrounding environment. Consequently, the top and bottom surfaces were designated as convective heat transfer boundaries, with detailed parameters presented in Table 1. The other walls were treated as adiabatic boundaries, assuming no heat transfer through them. This setup ensures a realistic simulation of the thermal management challenges in stacked 3D ICs.

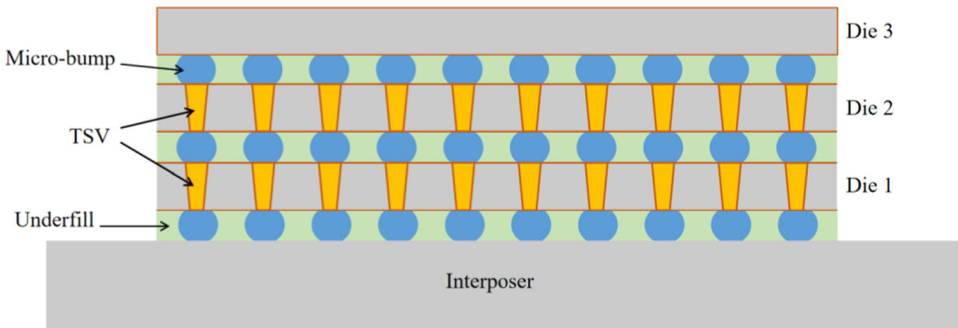


FIGURE 1 Three-layer stacked three-dimensional integrated circuits model.

TABLE 1 Parameters related to convective heat transfer boundary conditions.

	Top	Bottom
Convection heat transfer coefficient ($\text{Wm}^{-2}\text{K}^{-1}$)	35,000 ¹⁹	30
Ambient temperature ($^\circ\text{C}$)	5	23

In the context of a realistic 3D ICs model, heat sources are typically distributed across each layer of the chip, with varying sizes and power levels. To streamline the analysis, a degree of simplification of these heat sources is often necessary. In the present study, we have simplified the model to include four heat sources positioned within TSVs arrays. Specifically, one heat source is allocated to each of the first and second layers, while two heat sources are designated for the third layer. The heat sources are assigned powers of 0.1, 0.08, 0.06, and 0.1 W, respectively, and each has structural dimensions of $80\ \mu\text{m} \times 80\ \mu\text{m} \times 40\ \mu\text{m}$. Given that the placement of heat sources in chip design is typically constrained to specific regions, we have imposed limitations on the permissible variation in the location of each heat source. These constraints are delineated in Figure 2, ensuring that the heat source locations are confined within predetermined areas to reflect practical chip design considerations.

2.2 | Optimization objectives and parameters

The primary objective of optimization in thermal management of 3D ICs is to reduce the maximum temperature and improve the overall distribution of thermal conditions. This strategy is crucial for preventing localized overheating and the consequent degradation of local stress and heat dissipation conditions in areas characterized by high power density. In practical chip thermal design, it is imperative to consider multiple temperature metrics. In this paper, three temperature indicators were considered, which were the maximum temperature T_{max} , the average temperature T_{mean} , and the maximum temperature difference T_{diff} across the 3D ICs. While these indicators are indispensable for a comprehensive thermal assessment, they often present conflicting requirements. This inherent conflict underscores the multiobjective nature of the optimization problem in thermal design. To address this complexity, we transformed the multiobjective optimization problem into a single-objective formulation using a weighted average method. The objective function ($\min F$) was defined accordingly, allowing for a balanced consideration of all key temperature indicators within a unified framework. It is defined as

$$\min F = w_1 T_{\text{max}} + w_2 T_{\text{mean}} + w_3 T_{\text{diff}}, \quad (1)$$

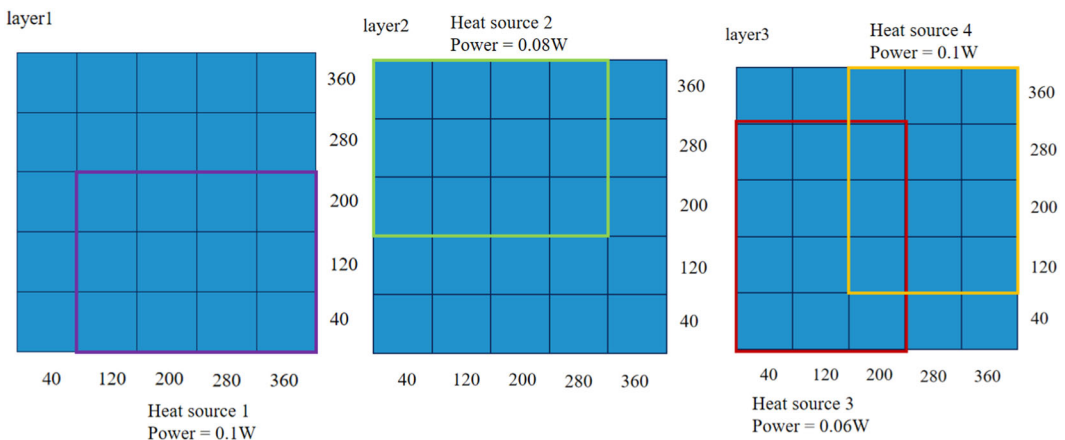


FIGURE 2 Range of heat source distribution per layer of Dies.

where w_1 , w_2 , and w_3 are weight coefficients. Combining multiple objectives as a weighted sum has some disadvantages, the optimization results strongly depend on the setting of weighting coefficients. If the weighting coefficients change significantly, the optimization calculations may need to be redone. However, this method also has the advantage of simplicity and efficiency. Once the weighting coefficients are determined, the optimization results can be obtained quickly and applied in practice immediately. Therefore, in the domain of chip thermal design, it is widely used by many scholars.¹⁶

To reduce the maximum temperature as the main objective of optimization, while taking into account the reduction of the average temperature and the maximum temperature difference, weight coefficients were set to be: $w_1 = 0.5$, $w_2 = w_3 = 0.25$. In addition, all temperature data were normalized to be dimensionless, thus mitigating the influence of the order of magnitude between data points and making data comparison more favorable. The formula for normalization is as follow²⁰:

$$T_n = \frac{T - T_{\min}}{T_{\max} - T_{\min}}, \quad (2)$$

where T_n is the normalized data, and T is the original data; $T_{\max}$ and $T_{\min}$ are the maximum and minimum temperature values in the data, respectively.

The optimization parameters were the horizontal and vertical coordinates of the center locations of the four heat sources, a total of eight parameters. The distribution of the center position of each heat source needed to satisfy restricting conditions, which can be referred to Figure 2.

2.3 | Structuring of neural network prediction model

In this study, we explore the impact of positional variations of four heat sources on the temperature field of a chip, with each source confined to a specified area. Achieving comprehensive coverage of sample points for this analysis necessitates significant computational resources. The complexity is further compounded when the number of heat sources is increased, leading to an exponential rise in the required number of samples. Given the computational demands associated with finite element simulations, it is imperative to minimize the simulation workload. The construction of a neural network predictive model presents an efficient alternative. By employing a neural network to approximate the simulation model, it is feasible to develop a surrogate model with high accuracy using a relatively small data set, thereby substantially reducing the need for extensive data sampling.

Key parameters that define the architecture of a feedforward neural network model include the size of the sample data set, the number of hidden layers and neurons, the choice of transfer function, and the training function. The conventional architecture of a feedforward ANN reduced-order model is depicted in Figure 3, providing a visual representation of the model's structure.

(1) Determining transfer functions, hidden layers and neurons

The operation of a BP neural network is fundamentally structured around two principal phases: forward propagation of the signal and backward propagation of the error. These phases are

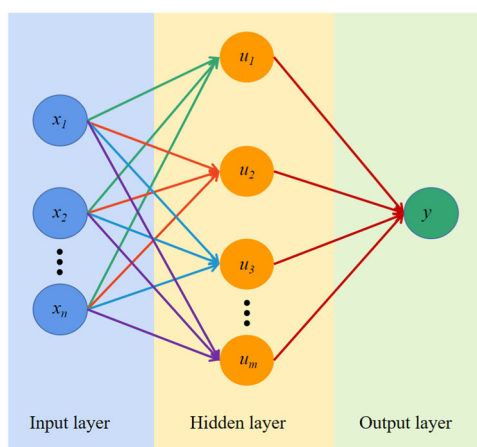


FIGURE 3 Components of a typical feed-forward neural network.

critical for the training process, enabling the network to adjust and refine its predictions. During the forward propagation phase, the network processes inputs through its layers up to the output layer using a transfer function. For our BP neural network model, we have selected the nonlinear Sigmoid function as the transfer function. This function is advantageous as it maps input values into a bounded (0,1) range, providing a smooth gradient necessary for the activation of neurons. The backward propagation phase is where the network updates its weights to minimize prediction errors. This adjustment is facilitated by the derivative of the Sigmoid function, which is employed to propagate the error signal backwards through the network. The derivative plays a crucial role as it influences the rate and effectiveness of weight adjustments during training. To quantify prediction errors, the MSE function is utilized as the loss function. The minimization of this loss function is achieved through the gradient descent method, an optimization technique that iteratively adjusts the network's weights by moving towards the steepest descent of the error landscape. Together, the use of the Sigmoid function for signal propagation, its derivative for error backpropagation, and the MSE for loss calculation constitute the core framework of our BP neural network model. This configuration aims to enhance the network's learning efficiency and improve its overall prediction accuracy.

The number of hidden layers in the neural network model is typically determined by the complexity of the model. Increasing the number of hidden layers could potentially enhance the model's prediction accuracy by capturing more complex relationships in the data. However, this increase in depth also leads to a significant escalation in model complexity, which can complicate the training process and increase the risk of overfitting, especially when the data set is not sufficiently large or diverse. For the specific application in this study, where the independent variable is defined as the heat source location coordinates and the dependent variable as the weighted temperature indicators, our analysis indicated a relatively straightforward relationship. The data exhibited a good degree of regularity, suggesting that the underlying patterns could be effectively captured with a simpler model architecture. Based on these considerations, we opted for a single hidden layer, which was found to provide satisfactory prediction accuracy while maintaining model simplicity and manageability.

The number of neurons in the hidden layer influences the model's learning capacity. Too few neurons may result in underfitting, while an excessive number of neurons can lead to overfitting. The number of neurons is generally derived from empirical formulas, and continuous experimentation and adjustment are necessary for specific models to select the optimal neuron count. The initial value for neuron adjustment is set according to the empirical formula for the number of neurons in the form:

$$N = \frac{N_s}{a(N_i + N_o)}, \quad (3)$$

where N_s represents the sample size of the training set, a is a constant with a value range of 2 to 10. N_i is the number of neurons in the input layer, N_o is the number of neurons in the output layer. Through ongoing attempts and adjustments, the final selection for the number of hidden layer neurons was 17 as the parameter for model establishment.

(2) Sampling scheme for data

This data-set was produced through a sampling process that utilizes the COMSOL application programming interface (API) with MATLAB. A three-layer stacked 3D IC model, constructed within the COMSOL Multiphysics 5.5 software, was interfaced with MATLAB. Various heat source positions were then selected within MATLAB for batch sample generation, while each sample was simulated once in COMSOL. To minimize the sample size without compromising accuracy, the data collection process incorporated Latin hypercube sampling (LHS).^{21–23} This method merges the benefits of random and stratified sampling. Initially, the sample space is uniformly divided into N segments, followed by the selection of a random value within each segment to represent the sampling point for that interval. This approach ensures comprehensive coverage of the sample space while maintaining the randomness of sampling.

Given that LHS is a form of stratified sampling, the extracted sample points are evenly distributed across the entire sample space. This method divides the range of each variable into uniform subintervals, each with an equal probability of being sampled. Random selection within these intervals ensures that the sample points are both comprehensive and representative, effectively minimizing the risk of selecting extreme cases. Each representative sample is likely to be found within a designated subinterval, and the random sampling of each subinterval substantially increases the likelihood of selecting any specific representative sample. Consequently, LHS is highly effective in capturing all representative samples within the sample space, thus enhancing the representativeness and reliability of the sampling process. To illustrate this methodology, Figure 4 demonstrates a typical two-dimensional implementation of LHS. In such a setup, each distinct element appears exactly once in any given row or column, ensuring that each variable's full range is sampled without repetition. This arrangement facilitates a thorough and unbiased exploration of the multidimensional sample space.

Although the partitioning method for sample point selection reduces the total number of points, it fails to encompass all potential heat source locations for the optimization function. Conversely, the finite element simulation approach, which involves numbering unit blocks, aligns the heat source with the finite element unit model blocks, thereby facilitating batch data generation. To address the issues arising from partition sampling, each unit block's position was divided into a combination of central coordinates and heat source dimensions. This

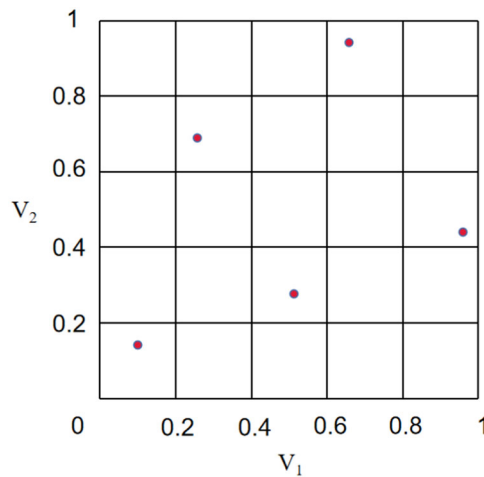


FIGURE 4 Typical two-dimensional Latin hypercube sampling.

strategy allowed for the selection of all possible heat source positions across the entire plane. Assuming a fixed square or rectangular heat source, the central coordinate variables of a single heat source were isolated, with x and y denoting the heat source's central position on the plane. The heat source's dimensions could be broken down into two variables: length (l_h) and width (w_h), thereby achieving the parameterization of a single heat source position. The goal of this parameterization was to enhance the resolution of heat source position selection issues and to allow for adjustments corresponding to heat sources of varying positions and shapes.

(3) Identification of sample size

A larger sample size enhances the accuracy of neural network training. However, generating a substantial amount of sample data for finite element simulation models consumes considerable computational resources, and excessive data may lead to overfitting, which in turn affects prediction accuracy. To minimize the sample size while ensuring fitting accuracy, LHS was employed, and modeling was performed with sample sizes of 100, 200, 500, 1728, and 10,000. A single hidden layer was chosen as the basis, and the Levenberg–Marquardt (L–M) algorithm was utilized as the training function for model establishment.

Since both input and output data in the neural network were normalized, the trained network was also oriented towards normalized data. To predict the output data, denormalization of the data was required. The function expression for denormalization was in the form:

$$y = (y_{\max} - y_{\min})y_n + y_{\min} \quad (4)$$

After completing the data normalization, the generated data set was organized, with 70% of the input data used for training, 15% for testing, and the remaining 15% for validation. The R coefficient and error histogram were selected as the evaluation criteria for sample data. The R coefficient is the overall regression value of the training, indicating the correlation between the predicted output value and the true output value. The range of R is between 0 and 1, where an R value closer to 1 signifies a stronger relationship between the predicted and output data, and

vice versa, a greater degree of randomness. The error histogram intuitively displays the deviation between the predicted and true output values, with the vertical axis reflecting the number of samples with corresponding deviation values.

As shown in Figure 5, the Pearson correlation coefficient R increases as the sample size of the trained model increases. When the sample size reaches 1728, the regression coefficients for the training, test, and validation sets all exceed 0.99, indicating a strong correlation within the model. A comparison of error histograms (in Figure 6) for sample sizes of 1728 and 10,000 reveals that both distributions are approximately normal and that errors fall within an acceptable range. Given its overall predictive performance, a data-set with a sample size of 1728 was ultimately selected for constructing the neural network reduced-order model.

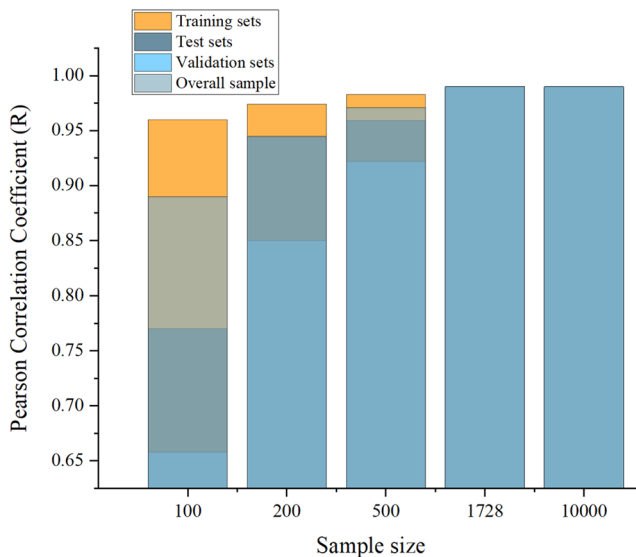


FIGURE 5 Effect of sample size on the R coefficient of the model.

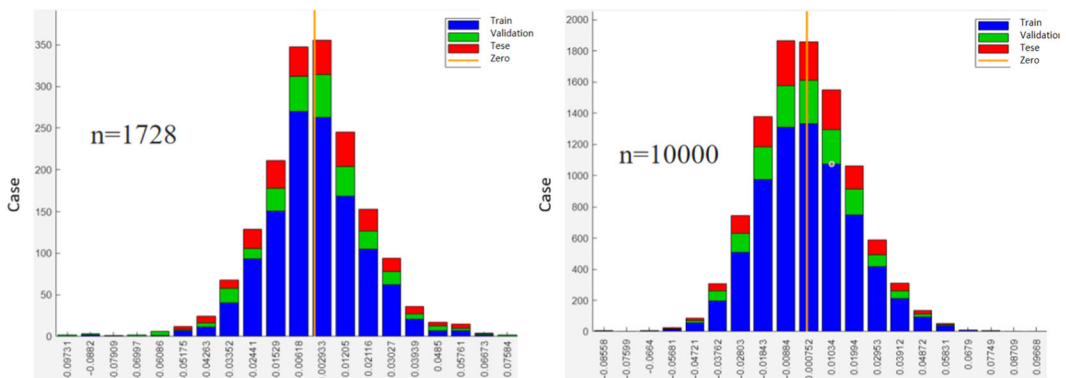


FIGURE 6 Comparison of error histograms at sample sizes of 1728 and 10,000.

(4) Selection of training functions

Numerous training functions exist for neural networks, including gradient descent, Newton's method, quasi-Newton's method, L–M algorithm, Bayesian regularization, and the quantized conjugate gradient method. To determine the most suitable training function for this model, the L–M algorithm, Bayesian regularization, and the quantized conjugate gradient method were evaluated based on prediction accuracy and computational speed.

The L–M algorithm, primarily employed for addressing nonlinear problems, amalgamates the benefits of the Gauss–Newton method and gradient descent.^{24,25} During the initial phase of iteration, the algorithm utilizes gradient descent, facilitating swift convergence when distant from the optimal value. As the iteration progresses, the rate of gradient descent significantly decelerates, and the Gauss–Newton method is employed for parameter estimation within a narrower range, yielding an estimated value proximate to the optimal one. The L–M algorithm's primary strength lies in its rapid convergence speed and broad applicability. However, its requirement to solve partial derivatives for each parameter estimation leads to a substantial increase in computation time when dealing with numerous input parameters. The quantized conjugate gradient method partitions the objective function into functions across multiple directions, continuously generating conjugate directions during iteration, and subsequently determining the extreme values of these functions independently. The final solution is then synthesized from these extreme values. This method only necessitates solving the first-order derivative, thereby circumventing the slow convergence typical of gradient descent in the later stages, and its computational complexity is less than that of Newton's method. This makes it a primary method for solving nonlinear optimization. However, it does have a notable drawback: while its solution method is relatively accurate and fast for quadratic functions, it does not offer a significant advantage in other scenarios.

The Bayesian regularization training method, grounded in Bayesian analysis,^{4,26,27} refines the error function by modulating the weight of the regularization term to optimize training performance. This algorithm is formulated by incorporating a network performance function, which is based on the L–M algorithm, and adjusting its weight. In neural networks, the error function is typically denoted by the mean square error (MSE)²⁸:

$$f_{MSE1} = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^m (P_i(j) - T_i(j))^2, \quad (5)$$

where $P_i(j)$ signifies the predicted value, $T_i(j)$ represents the actual value, and n denotes the total number of samples. The mean square error of the network weights, that is, the regularization term, is given by Equation (6):

$$f_{MSE2} = \frac{1}{n} \sum_{k=1}^n w_k^2, \quad (6)$$

where w_k is the weight of the network connection. The performance function, improved through this process, is $f_{BR} = w_1 \times f_{MSE1} + w_2 \times f_{MSE2}$, where w_1 and w_2 are regularization parameters. The ratios of w_1 and w_2 are determined through Bayesian analysis. The prior distribution of samples and network weights adheres to the Gaussian distribution. When the posterior function reaches its maximum value, f_{BR} is solved to obtain the optimal values of w_1 and w_2 :

$$w_1 = \frac{\varphi}{2f_{MSE2}(w)}, \quad (7)$$

$$w_2 = \frac{n - \varphi}{2f_{MSE1}(w)}, \quad (8)$$

where φ represents the number of effective weights, determined by Equation (9):

$$\varphi = M - w_1 * \text{trace}^{-1}(H), \quad (9)$$

where H is the Hessian matrix corresponding to the solution of w , given by $H = w_1 \nabla^2 f_{MSE1} + w_2 \nabla^2 f_{MSE2}$.

Bayesian regularization has the strongest generalization ability among the three algorithms and avoids overfitting and underfitting to some extent due to the determination of weights by Bayesian analysis. However, its calculation is more complex and requires more computational resources. Table 2 shows a comparison of the three training functions, where the R coefficient of the training algorithm using Bayesian regularization is the highest and the mean square error is the smallest. For the model being constructed, the model parameters are relatively simple and the fitting speed difference between various algorithms is not significant. To ensure the reliability of subsequent optimization results, Bayesian regularization with the best model accuracy is selected as the final training function, where the training set accounts for 80% and the test set accounts for 20%.

2.4 | Bayesian optimization

In various fields such as engineering, computer science, and economics, the task of identifying extrema of functions—which are typically continuous and exhibit smoothness properties like Lipschitz continuity—is common. However, these functions may lack explicit formulations or may be multimodal, with local extrema potentially yielding suboptimal outcomes in comparison to the global extremum. Optimizing such functions can be computationally demanding. A spectrum of optimization algorithms is available, ranging from gradient-based methods like gradient descent, Newton's method, quasi-Newton methods, and the conjugate gradient method, to heuristic approaches including PSO, SA, GA, and BO.²⁹ These algorithms are assessed against criteria such as computational efficiency and accuracy to determine their suitability for a given reduced-order model.

TABLE 2 Comparison of R -coefficients and MSE for the three training algorithms.

Training algorithm	R of the training sets	R of the validation sets	R of the test sets	MSE of the training sets	MSE of the validation sets	MSE of the test sets
L-M algorithm	0.9943	0.9908	0.9887	3.96e−4	7.12e−4	8.08e−4
Bayesian regularization	0.9952	\	0.9920	3.36e−4	\	5.59e−4
Conjugate gradient method	0.9802	0.9774	0.9773	0.0014	0.0015	0.0016

Global optimization, a subset of non-convex optimization, seeks the global extrema of a real-valued objective function through iterative point-wise evaluations. BO, which is predicated on Bayes' theorem and has been advocated for optimizing reduced-order models,^{17,30,31} follows a probabilistic approach as delineated in Equation (10):

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}, \quad (10)$$

where $P(h|D)$ represents the likelihood estimation, or the probability of D occurring given the hypothesis data, while $P(D|h)$ represents the posterior probability estimation, or the probability of h occurring given the prior data.

BO leverages the distribution of a prior function estimate and employs Bayesian inference to update this to a posterior distribution based on observed data. BO is underpinned by two principal elements: the probabilistic surrogate model, which encompasses the prior and the likelihood, and the acquisition function. The probabilistic surrogate model is responsible for generating the prior distribution $P(h)$ and the likelihood $P(D|h)$, which are integral to Bayes' theorem. The acquisition function, on the other hand, is formulated from the posterior probability distribution to minimize the expected loss across the evaluation points. Gaussian processes are frequently employed within BO as the surrogate model due to their proficiency in capturing nonlinearities and quantifying uncertainties. In this context, the unknown objective function $f(x)$ is modeled as a stochastic process, specifically a Gaussian process, which provides a robust framework for the optimization of complex functions in a variety of real-world applications, as shown in Equation (11):

$$f(x) \sim GP(\mu(x), \sigma(x)) = GP(\mu(x), H(x, x')), \quad (11)$$

where $\mu(x)$ represents the mean function of the objective function at point x , $\sigma(x)$ is the covariance function, and $H(x, x')$ is the kernel function. The Gaussian process is defined by a mean function and a covariance function, which are updated based on the points that have been evaluated. The mean function and covariance matrix can be obtained as follows:

$$\mu(x) = [\mu(x_1)\mu(x_2)\dots\mu(x_M)]^T, \quad (12)$$

$$\sigma(x) = \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_M) \\ \vdots & \ddots & \vdots \\ k(x_M, x_1) & \cdots & k(x_M, x_M) \end{bmatrix}, \quad (13)$$

where x_M is the M th observation point, and the k in the covariance matrix is defined by Equation (14):

$$k(x_i, x_j) = e^{\left(-\frac{1}{2}\|x_j - x_i\|^2\right)}. \quad (14)$$

To calculate the value of the next data point, consider the joint distribution of the new and old data points on f :

$$\begin{bmatrix} f(x) \\ f(x_{M+1}) \end{bmatrix} \sim N\left(\begin{bmatrix} m(x) \\ m(x_{M+1}) \end{bmatrix}, \begin{bmatrix} \sigma(x) & k \\ k^T & k(x_{M+1}, x_{M+1}) \end{bmatrix}\right). \quad (15)$$

From this, the mean $\mu(x_{M+1})$ and variance S^2 of $f(x_{M+1})$ can be calculated by Equations (16) and (17), respectively:

$$\mu(x_{M+1}) = k^T \sigma(x)^{-1} f(x), \quad (16)$$

$$S^2(x_{M+1}) = k(x_{M+1}, x_{M+1}) - k^T \sigma(x)^{-1} k. \quad (17)$$

This method calculates the posterior distribution of $f(x)$ from the current observation data and determines the next search point using the acquisition function based on Gaussian distribution information. The Bayesian part of the optimization comes from the use of Bayes' theorem to update the probabilistic model based on new data. This allows the method to incorporate all the information that has been gathered during the optimization process, making it a very efficient method for optimizing expensive functions. The BO process involves the following steps:

- (1) Define the required $f(x)$ and the domain of x .
- (2) Take a finite number of n values of x and calculate $f(x)$.
- (3) Estimate the objective function based on the finite observation values and obtain the target value on the estimated function f^* .
- (4) Define the rules and determine the next observation point.

Repeat steps (2)–(4) until a predefined stopping criterion is met, such as reaching a maximum number of iterations or exhausting computational resources. The fundamental concept underlying BO involves guiding the search process through the creation of a probabilistic model of the objective function. This model aids in identifying the parameter configuration that allows the objective function to attain a globally optimal solution.

3 | RESULTS AND DISCUSSION

3.1 | Methodological process validation

The effectiveness of the optimization process presented in this paper was corroborated using the models developed by Wang et al.¹⁶ These models provided a basis for a comparative analysis of chip layout optimization, with variations in power and number of heat sources, as illustrated in Figure 7. The optimization goal was to minimize the highest junction temperature across the chips. The results of the optimized layout were in close agreement with those obtained by Wang et al.,¹⁶ exhibiting a discrepancy of less than 1% in the highest junction temperature when compared through finite element simulation. This level of agreement indicates that the proposed optimization method possesses a high degree of universality and reliability. The permissible range for chip placement is indicated by red dashed lines in the figure, with the chip power values specified as follows: for two heat sources, power = [0.3, 0.3 W]; for three heat sources, power = [0.3, 0.3, 0.4 W]; and for four heat sources, power = [0.3, 0.3, 0.4, 0.4 W].

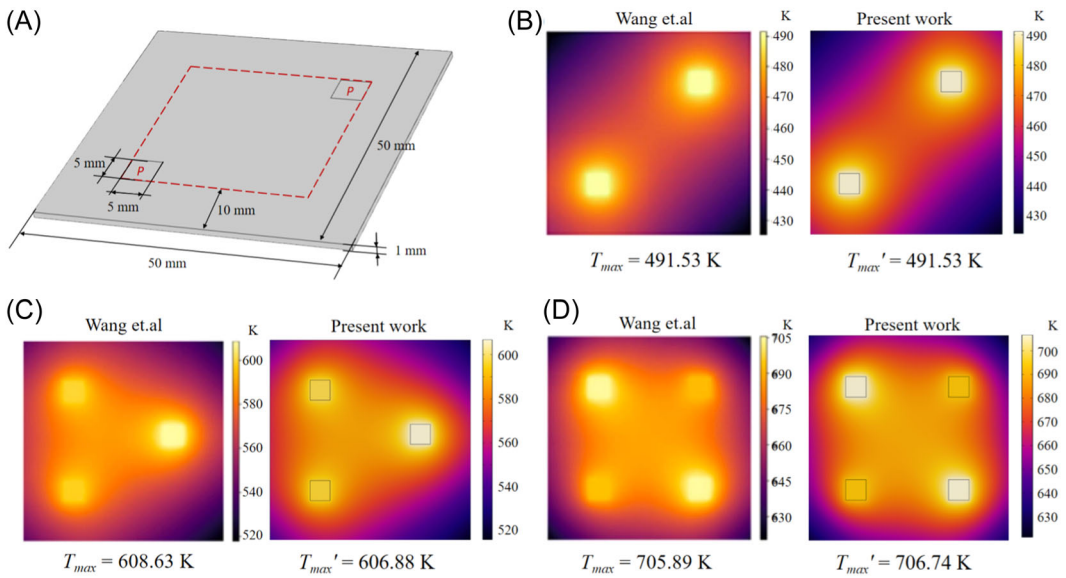


FIGURE 7 Comparison of Wang et al.'s optimization model and the optimization results and temperature distribution for different numbers of heat sources. (A) Wang et al.'s optimization model, (B) optimization results and temperature distribution for two heat sources, (C) optimization results and temperature distribution for three heat sources, (D) optimization results and temperature distribution for four heat sources.

3.2 | Performance of the ANN reduced-order model

The finalized parameters for the feedforward ANN reduced-order model have been established, comprising a single hidden layer with 17 neurons, a training set of 1728 samples, and the application of the Bayesian regularization training algorithm. To evaluate the neural network model's predictive accuracy, eight distinct sets of heat source data were randomly selected via Latin hypercube sampling. These datasets facilitated a comparison between the neural network's predictions and the results from finite element simulations. Additionally, the temperature distribution from a representative simulation model was scrutinized.

Table 3 lists the coordinates for the center positions of the heat sources for the eight chosen sample sets. The notation H_x^i denotes the horizontal coordinate of the i th heat source (i ranging from 1 to 4). The center position of each heat source is defined by a pair of x and y coordinates, culminating in eight input coordinates for the neural network model. The output of the model is quantified by the selected objective function (F), which encapsulates the thermal performance metrics of interest.

Figure 8 presents a comparison between the model prediction results for representative samples and the corresponding finite element simulation outcomes. The deviations in all eight instances are less than 2%, the coefficient of determination R^2 varies from 0.925 to 0.933, suggesting a high degree of accuracy in the prediction model. Furthermore, the neural network training speed is notably swift, demonstrating an effective balance between speed and accuracy. This underscores the appropriateness of the chosen model parameters.

TABLE 3 Parameterized coordinates of the eight representative sample groups selected for sampling.

Sample	H_x^1	H_y^1	H_x^2	H_y^2	H_x^3	H_y^3	H_x^4	H_y^4
Case 1	200	120	120	360	120	120	360	120
Case 2	360	40	40	360	40	200	280	280
Case 3	360	200	200	200	40	40	360	280
Case 4	120	200	200	360	120	120	200	360
Case 5	280	120	40	280	200	200	200	120
Case 6	200	200	200	280	40	280	280	360
Case 7	120	40	40	280	200	40	280	200
Case 8	200	40	120	200	120	280	360	200

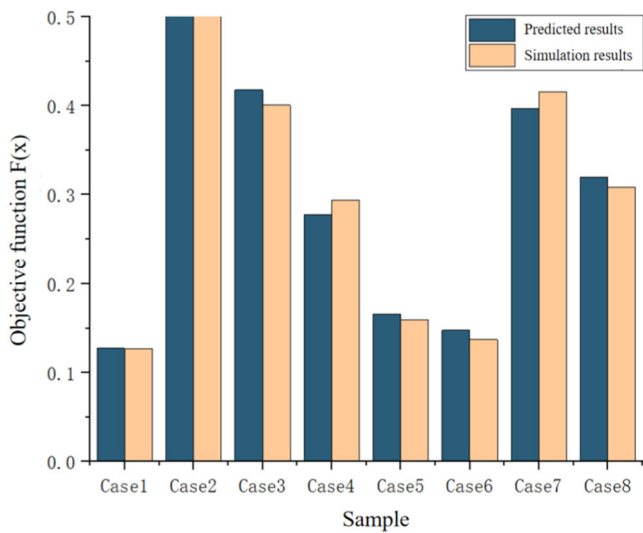


FIGURE 8 Comparison of model prediction results and finite element simulation results for representative samples.

In the context of Case 1, Figure 9 illustrates the temperature distribution across each layer of the model. The Die closest to the heat sink exhibits the lowest average temperature, while the Die with the highest power heat source displays the most significant temperature variance. Die 3, situated at the top and benefiting from optimal heat dissipation conditions, maintains the lowest average temperature. This suggests that positioning the heat source on Die 3, which has the shortest heat dissipation path, can result in a lower maximum temperature. Moreover, a more uniform temperature distribution across the Die is observed when the two hotspots are further apart. Consequently, it is advisable to avoid clustering multiple heat sources to prevent an excessively high local power density. Such a scenario could lead to an uneven chip temperature distribution, potentially resulting in localized overheating and excessive thermal stress.

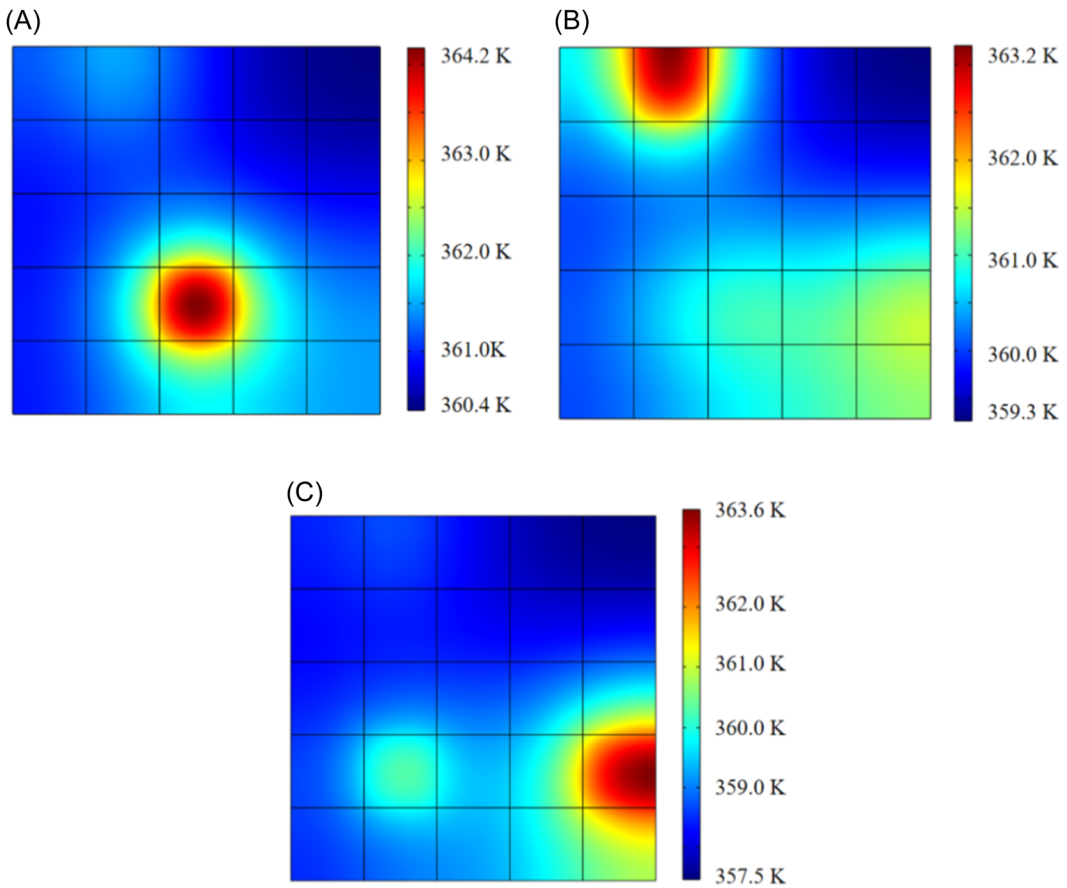


FIGURE 9 Temperature distribution per layer of Dies for the Case 1. (A) Die 1, (B) Die 2, (C) Die 3.

3.3 | Validation and application of Bayesian optimization

To ascertain the most effective heat source layout and optimization algorithm for the model, various optimization algorithms were implemented and comparatively analyzed. The optimal heat source configurations derived from each algorithm were identified, and their precision and computational resource demands were evaluated. All calculations were conducted on a laptop configured with an AMD Ryzen 9-4900H CPU and 16 GB of RAM, utilizing MATLAB R2022a software. This software suite facilitated the implementation of seven distinct algorithms, including Bayesian optimization, configured with standard hyperparameters as referenced in Otaki et al.¹⁵

For the GA algorithm, a population size of 50 was chosen, with “Stochunif”—a stochastic uniform selection function—governing the selection process. The mutation and crossover probabilities were set at 0.01 and 0.8, respectively. The PSO algorithm was also configured with a population size of 50, and both cognitive and social learning factors (C_1 and C_2) were set at 1.49. The SA algorithm's parameters included a defined number of iterations per temperature and a temperature reduction coefficient of 0.99. The execution of additional algorithms such as gradient descent, grid search, and surrogate optimization was enabled by MATLAB's

optimization toolbox, which provides a comprehensive suite of tools for solving complex optimization problems.

The constructed model was optimized using the bayesopt function in MATLAB, with the center of the Base Die serving as the reference point. The optimal position parameters of the three heat sources, as determined by the algorithms, are presented in Table 4. After 250 iterations, the algorithm yielded the best objective function output $F(x) = 0.0252$. These coordinates were then integrated into the constructed finite element model, and a comparative verification was conducted using COMSOL software. The final result, obtained through finite element calculation, was $F(x)' = 0.0273$, exhibiting an overall calculation deviation of less than 0.003, indicative of high accuracy. A horizontal cross-section of Die 3, housing two heat sources, was chosen for analysis. Figure 10 illustrates its temperature distribution, with the highest and lowest temperatures recorded at 363.4 and 357.7 K, respectively.

Table 5 illustrates a comparative analysis of the optimization results and parameters for seven different algorithms applied to the reduced-order model. The results reveal that the BO, GA, and surrogateopt algorithms successfully identify the optimal parameters and achieve the minimum value of the objective function. In contrast, the other algorithms do not perform as effectively, often converging to local optima and failing to find the global optimum within the allocated number of iterations. These less successful algorithms also tend to require more time to complete the optimization computations. Among the three algorithms that reached the global optimum, the BO algorithm stands out as the most efficient. It successfully locates the global optimum in just 4.07 s and requires only 250 iterations. In comparison, the surrogateopt algorithm, while effective in finding the optimum, is considerably more time-consuming, taking 64 s and 470 iterations to complete the optimization. The GA algorithm, although quicker than surrogateopt at

TABLE 4 Coordinates of the center points of the three heat sources obtained by Bayesian optimization.

Coordinate	Heat source 1	Heat source 2	Heat source 3	Heat source 4
$X (\mu\text{m})$	207.075	101.757	200	360
$Y (\mu\text{m})$	174.732	316.325	40	252

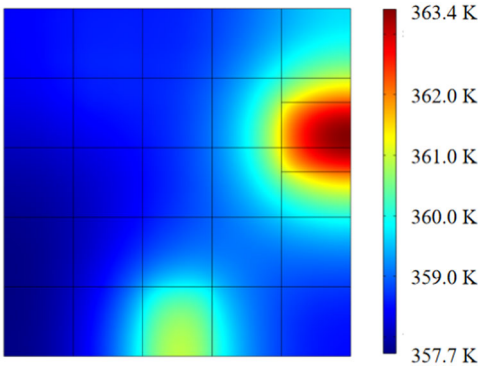


FIGURE 10 Temperature distribution of Die 3 after Bayesian optimization.

TABLE 5 Comparison of results of optimization algorithms.

Parameter	BO	GA	PSO	SA	Gradient descent	Grid search	Surrogateopt
H_x^1	207.075	206.789	215.997	210.651	236.167	236.178	206.286
H_y^1	174.732	174.542	160.65	174.417	162.928	162.926	174.047
H_x^2	101.757	103.492	142.683	117.717	69.689	96.692	104.152
H_y^2	316.325	317.017	264.856	319.380	274.262	274.21	317.339
H_x^3	200	200	141.836	138.136	200	200	200
H_y^3	40	40	182.063	41.37	40	40	40
H_x^4	360	360	339.209	360	247.114	247.084	360
H_y^4	252	252.11	263.601	252.437	243.837	243.841	252.06
$F(x)$	0.0252	0.0252	0.0316	0.0263	0.0317	0.0317	0.0252
Runtime (s)	4.07	5.83	17.7	20.5	21.8	23.2	64

5.83 s, requires about 2500 iterations to locate the optimal point, which is significantly more than that required by the BO algorithm. This detailed comparison underscores the efficiency and effectiveness of the BO algorithm in this specific application.

The GA algorithm's computation time is significantly influenced by its population size, which involves evaluating 50 layout patterns per iteration, in contrast to the BO evaluation of a single layout pattern. This difference results in GA consuming a greater amount of computational resources than BO. Further adjustments to the GA's population size revealed that reducing the number of patterns per iteration hinders the algorithm's ability to locate the optimal value within the designated time frame. Given the objective of swiftly achieving an optimal chip layout that minimizes the model's temperature distribution, the BO algorithm emerges as the faster and more resource-efficient method of optimization for this context. This efficiency, along with its reduced computational resource demand, contributes to the popularity of BO in hyperparameter optimization.

As illustrated in Figure 11, the temperature distribution between layers is more uniform when using the BO algorithm compared to Case 1, which represented the best objective function value among randomly selected samples. $T_{\max}$ is slightly reduced, and the distribution of heat sources across each layer's Die is more even. When the heat source positions from each layer's Die are projected onto the same plane, there is no overlapping of heat sources, allowing each layer's Die to maintain the shortest possible heat dissipation path. This arrangement significantly reduces the overall junction temperature. These optimization results align well with practical scenarios. In future work, as the number of heat sources increases for optimal chip design, there will be an exponential increase in computational demands. Compared with other optimization algorithms, the BO algorithm offers significant advantages in enhancing computational efficiency and conserving resources, making it particularly suitable for complex optimization tasks in 3D ICs design.

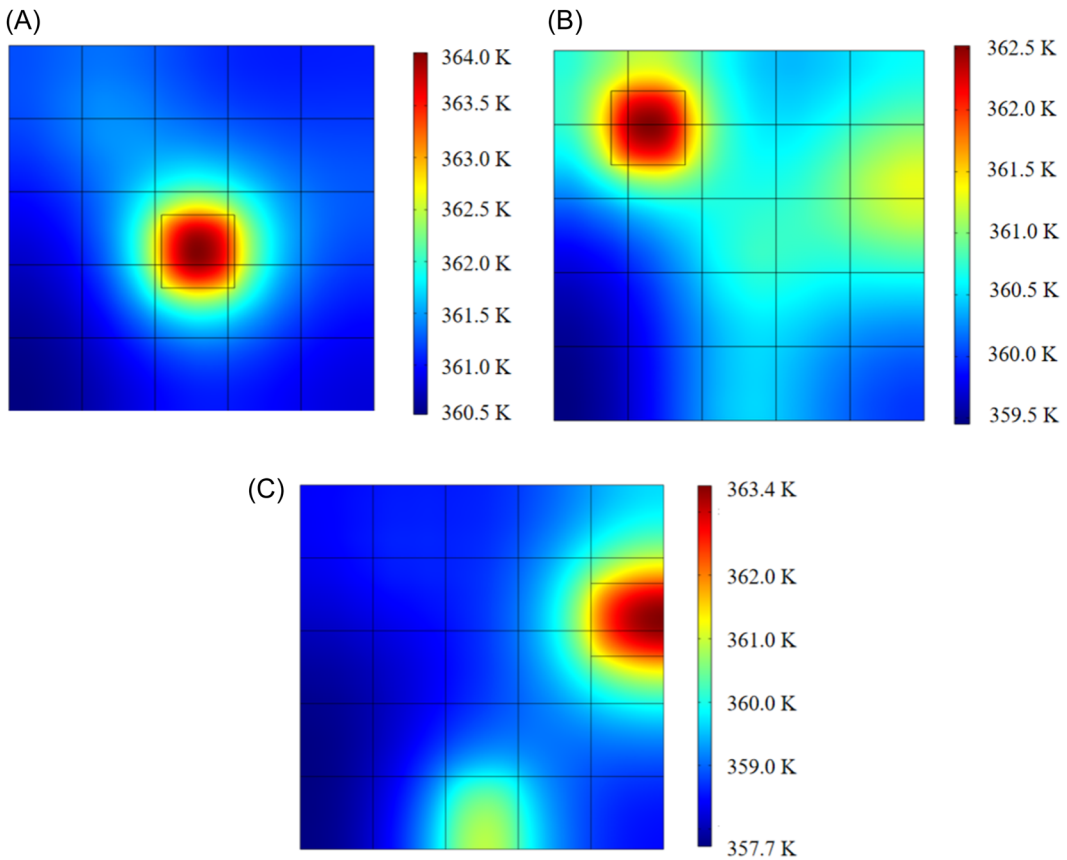


FIGURE 11 Temperature distribution of Die in each layer after optimization. (A) Die 1, (B) Die 2, (C) Die 3.

4 | CONCLUSIONS

In this study, we have employed a methodology that combines ANN reduced-order model with BO to predict and optimize the temperature distribution in 3D ICs with variable heat source layouts. We have developed a three-layer stacked 3D ICs model, which allows for the parametric positioning of heat sources. The use of LHS has been instrumental in reducing the computational data requirements for this study. The COMSOL Multiphysics API, interfaced with MATLAB, facilitated the batch generation of the necessary training data. A mathematical model was then formulated to predict the effects of heat source placement on the temperature distribution within the 3D ICs. The trained reduced-order model underwent global optimization via BO to ascertain the optimal parameters for heat source positioning. The predictive results were subsequently cross-verified with actual finite element simulation outcomes to confirm the precision of the prediction model and the efficacy of the optimization algorithm. For the 3D ICs model constructed in this study, BO identified the global optimum within a remarkably short time frame of 4.07 s over 250 iterations. In comparison, the surrogateopt algorithm required 64 s and 470 iterations to complete the optimization process. The GA, while faster than surrogateopt at 5.83 s, demanded a considerably higher number of iterations, totaling 2500, to pinpoint the

optimal solution. It is noteworthy that the GA evaluates 50 layout patterns per iteration, as opposed to BO's single evaluation, which underscores BO's lower computational resource consumption.

Although the prediction and optimization process developed in this study effectively determines the optimal distribution of heat source positions in 3D ICs, the optimization of the reduced-order model still requires the generation of a substantial amount of simulation data. Future research could focus on devising methods for constructing reduced-order models that minimize the resources needed for data generation and enhance the accuracy of temperature distribution predictions in 3D ICs. The current study is limited to examining the effects of heat source positioning on the overall temperature distribution of 3D ICs. Subsequent research could expand the scope to include the impact of varying material property parameters, thereby advancing thermal management strategies for 3D ICs.

ACKNOWLEDGMENTS

This work was supported by the Natural Science Foundation of China (Grant Number 52006095).

CONFLICT OF INTEREST STATEMENT

The authors declare no conflict of interest.

DATA AVAILABILITY STATEMENT

No data was used for the research described in the article.

ORCID

Xixin Rao  <http://orcid.org/0000-0002-3936-5918>

REFERENCES

1. Xiao C, He H, Li J, Cao S, Zhu W. An effective and efficient numerical method for thermal management in 3D stacked integrated circuits. *Appl Therm Eng.* 2017;121:200-209.
2. Wang C, Huang XJ, Vafai K. Analysis of hotspots and cooling strategy for multilayer three-dimensional integrated circuits. *Appl Therm Eng.* 2021;186:116336.
3. Zhang H, Che FX, Lin TY, et al. *Modeling, Analysis, Design, and Tests for Electronics Packaging beyond Moore*. Chemical Industry Press Co., Ltd; 2019.
4. Chen Y, Li D, Feng S, Huang Q, Chen Z, Shu D. Optimization and thermal-performance deep learning on carbon/epoxy composite panels with microchannel structure for battery cooling. *Appl Therm Eng.* 2022; 217:119162.
5. Sikirica A, Grbčić L, Kranjčević L. Machine learning based surrogate models for microchannel heat sink optimization. *Appl Therm Eng.* 2023;222:119917.
6. Hua Y, Yu CH, Zhao Q, Li MG, Wu WT, Wu P. Surrogate modeling of heat transfers of nanofluids in absorbent tubes with fins based on deep convolutional neural network. *Int J Heat Mass Transfer.* 2023; 202:123736.
7. Abdelghany MT, Elshamy SM, Sharafeldin MA, Abdellatif OE. Utilizing artificial neural networks to predict the thermal performance of conical tubes with pulsating flow. *Appl Therm Eng.* 2023;224:120087.
8. Qian X, Xuan D, Zhao X, Shi Z. Heat dissipation optimization of lithium-ion battery pack based on neural networks. *Appl Therm Eng.* 2019;162:114289.
9. Fakheri Tabrizi A, Behjat L, Swartz W, Rakai L. A fast force-directed simulated annealing for 3D IC partitioning. *Integration.* 2016;55:202-211.

10. Guo Z, Yang B, Chen Y, et al. Modular thermoelectric generation arrays reconfiguration under heterogeneous temperature distribution via improved cooperation search algorithm: modelling, design and HIL validation. *Appl Therm Eng*. 2023;219:119323.
11. Nguyen NP, Maghsoudi E, Roberts SN, Kwon B. Shape optimization of pin fin array in a cooling channel using genetic algorithm and machine learning. *Int J Heat Mass Transfer*. 2023;202:123769.
12. Manaserh M, Gharaibeh AR, Tradat MI, Rangarajan S, Sammakia BG, Alissa HA. Multi-objective optimization of 3D printed liquid cooled heat sink with guide vanes for targeting hotspots in high heat flux electronics. *Int J Heat Mass Transfer*. 2022;184:122287.
13. Ni T, Chang H, Zhu S, et al. Temperature-aware floorplanning for fixed-outline 3D ICs. *IEEE Access*. 2019;7:139787-139794.
14. Ren Z, Alqahtani A, Bagherzadeh N, Lee J. Thermal TSV optimization and hierarchical floorplanning for 3-D integrated circuits. *IEEE Trans Compon Packag Manuf Technol*. 2020;10:599-610.
15. Otaki D, Nonaka H, Yamada N. Thermal design optimization of electronic circuit board layout with transient heating chips by using Bayesian optimization and thermal network model. *Int J Heat Mass Transfer*. 2022;184:122263.
16. Wang ZQ, Hua Y, Aubry N, Zhou ZF, Feng F, Wu WT. Fast optimization of multichip modules using deep learning coupled with Bayesian method. *Int Commun Heat Mass Transfer*. 2023;141:106592.
17. Park SJ, Bae B, Kim J, Swaminathan M. Application of machine learning for optimization of 3-D integrated circuits and systems. *IEEE Trans Very Large Scale Integration Syst*. 2017;25:1856-1865.
18. Rao X, Song J, Tian Q, Liu H, Jin C, Xiao C. A universal and efficient equivalent modeling method for thermal analysis of 3D ICs containing tapered TSVs. *Int Commun Heat Mass Transfer*. 2022;136:106161.
19. Elliott JW, Lebon MT, Robinson AJ. Optimising integrated heat spreaders with distributed heat transfer coefficients: a case study for CPU cooling. *Case Stud Thermal Eng*. 2022;38:102354.
20. Polat ME, Cadirci S. Artificial neural network model and multi-objective optimization of microchannel heat sinks with diamond-shaped pin fins. *Int J Heat Mass Transfer*. 2022;194:123015.
21. Masoomzadeh M, Charkhtab Basim M, Chenaghlou MR, Khajehsaeid H. Probabilistic performance assessment of eccentric braced frames using artificial neural networks combined with correlation latin hypercube sampling. *Structures*. 2023;48:226-240.
22. Bayar G, Hambarci G. Improving measurement accuracy of indoor positioning system of a mecanum wheeled mobile robot using Monte Carlo—Latin hypercube sampling based machine learning algorithm. *J Franklin Inst*. 2023;360:13994-14021.
23. Rangarajan S, Choobineh L, Sammakia B. Supervised machine-learning approach for the optimal arrangement of active hotspots in 3-D integrated circuits. *IEEE Trans Compon Packag Manuf Technol*. 2021;11:1724-1733.
24. Hemmat Esfe M, Amoozadkhalili F, Toghraie D. Determining the optimal structure for accurate estimation of the dynamic viscosity of oil-based hybrid nanofluid containing MgO and MWCNTs nanoparticles using multilayer perceptron neural networks with Levenberg–Marquardt algorithm. *Powder Technol*. 2023;415:118085.
25. Zhang K, Zhang Z, Han Y, Gu Y, Qiu Q, Zhu X. Artificial neural network modeling for steam ejector design. *Appl Therm Eng*. 2022;204:117939.
26. Ma X, Xie M, Suykens JAK. A novel neural grey system model with Bayesian regularization and its applications. *Neurocomputing*. 2021;456:61-75.
27. Sabir Z, Akkurt N, Ben Said S. A novel radial basis Bayesian regularization deep neural network for the Maxwell nanofluid applied on the Buongiorno model. *Arab J Chem*. 2023;16:104706.
28. Tausendschön J, Radl S. Deep neural network-based heat radiation modelling between particles and between walls and particles. *Int J Heat Mass Transfer*. 2021;177:121557.
29. Roy S, Banerjee S. 3D IC partitioning approach for achieving lower TSV count and reduced heat dissipation using particle swarm optimization. In: *First International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*, Trichy, India, 2022:1-5.
30. Chen Y, Chen H, Zeng H, et al. Structural optimization design of sinusoidal wavy plate fin heat sink with crosscut by Bayesian optimization. *Appl Therm Eng*. 2022;213:118755.

31. Wang Y, Kandeal AW, Swidan A, et al. Prediction of tubular solar still performance by machine learning integrated with Bayesian optimization algorithm. *Appl Therm Eng*. 2021;184:116233.

How to cite this article: Zhang H, Song J, Rao X, Liu H, Xiao C. Rapid heat source layout optimization in three-dimensional integrated circuits using artificial neural network reduced-order model in combination with Bayesian optimization. *Heat Transfer*. 2024;53:3409-3431. doi:10.1002/htj.23095